1　解数独问题

1.1　问题分析

1. 问题描述：给定一个 9×9 的部分填充数独棋盘，其中空白格用字符 '.' 表示，需要在每个空白格填入数字 '1' 到 '9'，使得每行、每列以及每个 3×3 的子宫格内数字 1–9 均恰好出现一次。标准的数独问题保证输入有唯一解。

2. 输入格式：一个 9×9 的二维字符向量，已知数字用对应的字符表示，未知格用 '.' 填充。

3. 输出要求：在原棋盘上直接修改，将所有 '.' 替换为合法的数字，使得整个棋盘满足数独规则。

4. 问题关键点

（1）状态空间巨大：最多有 81 个格子，每个空白格有 9 种可能，朴素暴力枚举会面临组合爆炸。

（2）约束条件明确：行、列、宫格三条强约束可大幅剪枝。

（3）需要一种系统性的搜索策略，在满足约束的前提下尝试填写，并在发现矛盾时回退。

1.2　算法思想

（一）核心思路：回溯法（Backtracking）

1. 遍历棋盘寻找第一个空白格：按行优先顺序扫描，找到第一个内容为 '.' 的格子。

2. 尝试填入合法数字：对该空白格依次尝试 '1' 到 '9'，对每个候选数字，调用 isvalid 检查其在当前棋盘状态下是否满足行、列、3×3 宫格无冲突。

3. 递归深入：若候选数字合法，则填入并递归调用自身，尝试求解剩余空白格。

4. 回溯：如果递归调用返回 false（说明后续填入无法完成），则撤销当前填入（恢复为 '.'），继续尝试下一个候选数字。

5. 终止条件

（1）若所有格子均已填满（未找到 '.'），返回 true，表示找到解。

（2）若某空白格所有 1–9 尝试后均失败，返回 false，触发上层回溯。

（二）合法性检查函数 isvalid

1.　检查当前列 col 是否已有相同数字；

2. 检查当前行 row 是否已有相同数字；

3. 定位所在 3×3 宫格的左上角 (boxrow, boxcol)，检查宫格内是否已有相同数字

若上述三者均无，则返回 true。

1.3 算法伪代码

```
函数 isvalid(board, num, row, col):
    对于 i 从 0 到 8:
        如果 board[i][col] == num 则返回 false
    对于 j 从 0 到 8:
        如果 board[row][j] == num 则返回 false
    boxrow = (row / 3) * 3
    boxcol = (col / 3) * 3
    对于 i 从 boxrow 到 boxrow+2:
        对于 j 从 boxcol 到 boxcol+2:
            如果 board[i][j] == num 则返回 false
    返回 true


函数 backtrack(board):
    对于 i 从 0 到 8:
        对于 j 从 0 到 8:
            如果 board[i][j] == '.':
                对于 num 从 '1' 到 '9':
                    如果 isvalid(board, num, i, j):
                        board[i][j] = num
                        如果 backtrack(board) 返回 true:
                            返回 true
                        board[i][j] = '.'    // 回溯
                返回 false   // 所有数字均失败
    返回 true   // 未发现空白格，已求解
```

过程 game(board):

调用 backtrack(board)

主程序 main:

初始化 board 为给定 9×9 字符矩阵

game(board)

调用 printBoard(board) 输出结果

1.4 复杂度分析

令数独棋盘大小为固定常数 9×9，其中空白格数量记为 k（$0 \le k \le 81$）。

（一）时间复杂度

1. 最坏情况下，每个空白格尝试 9 个数字，回溯树的最大深度为 k，分支因子 ≤9，因此时间复杂度上界为 $O(9^k)$。

2. 实际上强约束剪枝极大降低了搜索量：每行、每列、每宫最多只剩 9 个数字，且随着填入增加，可选数字急剧减少。对于标准唯一解数独，回溯调用的总次数通常仅在数百到数千量级，远小于最坏上界。

3. 每次递归调用中，isvalid 检查固定执行最多 9+9+9=27 次比较，为 $O(1)$。

4. 总体实际运行时间对任何合法 9×9 数独均可瞬间完成。

（二）空间复杂度

1. 递归深度最大为 $k \le 81$，递归调用栈占用 $O(k)$ 空间。

2. 棋盘本身使用固定大小的二维向量，额外空间为 $O(1)$（忽略输入存储）。

3. 整体空间复杂度为 $O(k)$，最坏情况 $O(81)$，可视为 $O(1)$。

2 24 点游戏

2.1 问题分析

1. 问题描述：给定四个整数（通常为 1~10 的扑克牌点数），允许使用加（+）、减（-）、乘（*）、除（/）以及括号改变运算顺序，判断能否通过这四个数字的一次排列和合理组合，使表达式的结果等于 24。每个数字必须恰好使用一次。

2. 输入格式：四个整数，由调用者传入并转换为双精度浮点数向量。

3. 输出要求：若存在至少一种运算方式得到 24，则返回 true；否则返回 false。本代码在 main 中以布尔形式打印结果。

4. 问题关键点

（1）四个数字的运算顺序和括号组合方式繁多，需要系统化地穷举所有可能。

（2）中间结果可能为分数（如 8÷3 等），必须保留足够精度，最终与 24 比较时需考虑浮点误差。

（3）减法与除法不满足交换律，因此两个数的顺序会产生不同结果，必须分别考虑 a−b, b−a 和 a÷b, b÷a。

2.2 算法思想

核心思路：递归回溯穷举二元运算组合

1. 递归终止条件：若当前数字列表中仅剩一个数，判断其是否近似等于 24（使用 fabs(x - 24) < 1e-6 处理浮点精度）。若成立则返回 true，表示找到解。

2. 枚举所有二元操作：从当前列表中任选两个**不同**的数字 a 和 b（有序选择，即 i≠j），计算六种可能的运算结果：a + b，a – b，b – a，a * b，a / b （b≠0），b / a （a≠0）

3. 生成新数字集合并递归：将未选中的其余数字收集到新列表 restnum 中，然后将当前运算结果推入 restnum，作为“新数字”替换原有的 a 和 b。递归调用回溯函数，若返回 true 则逐层向上返回 true，否则撤销该运算（弹出结果），尝试下一种运算。

4. 回溯特性：若某个数字状态下的所有两数组合及所有六种运算均无法达到 24，则返回 false，上层递归继续尝试其他可能性。由于每次运算减少一个数字，递归深度恰好为初始数字个数减一（当初始 4 个数时深度为 3）。

补充：为何使用 double：整数运算过程中可能产生无限小数（如 1÷3），使用双精度浮点数并设置容差 1e-6，能正确判断如 8 ÷ (3 - 8 ÷ 3) = 24 这类表达式。

2.3 算法伪代码

```
函数 backtrack(nums: 双精度浮点数向量):
    如果 nums.size() == 1:
        返回 |nums[0] - 24| < 1e-6
```

```
n = nums.size()
对于 i 从 0 到 n-1:
    对于 j 从 0 到 n-1:
        如果 i == j 则跳过
        a = nums[i]
        b = nums[j]

        restnum = []
        对于 k 从 0 到 n-1:
            如果 k != i 且 k != j:
                restnum.push_back(nums[k])

        // 尝试 a + b
        restnum.push_back(a + b)
        如果 backtrack(restnum) 为真: 返回真
        restnum.pop_back()

        // 尝试 a - b
        restnum.push_back(a - b)
        如果 backtrack(restnum) 为真: 返回真
        restnum.pop_back()

        // 尝试 b - a
        restnum.push_back(b - a)
        如果 backtrack(restnum) 为真: 返回真
        restnum.pop_back()

        // 尝试 a * b
```

```
            restnum.push_back(a * b)
            如果 backtrack(restnum) 为真: 返回真
            restnum.pop_back()

            // 尝试 a / b (b 非零)
            如果 |b| > 1e-6:
                restnum.push_back(a / b)
                如果 backtrack(restnum) 为真: 返回真
                restnum.pop_back()

            // 尝试 b / a (a 非零)
            如果 |a| > 1e-6:
                restnum.push_back(b / a)
                如果 backtrack(restnum) 为真: 返回真
                restnum.pop_back()

        返回 假

    函数 judge(nums):
        返回 backtrack(nums)

    主程序:
        cards1 = [4,1,8,7] -> nums1
        cards2 = [1,2,1,2] -> nums2
        输出 judge(nums1), judge(nums2)
```

2.4 复杂度分析

令初始数字个数为 n（本题中 $n = 4$）。

（一）时间复杂度

1. 每一层递归中，从 k 个数字中选择两个有序对，有 $P(k,2)=k(k-1)$ 种选法。每种选法尝试最多 6 种运算。

2. 因此，从 n 个数逐步减少至 1 个数的总调用次数上界为：

$$T(n)=\prod_{k=2}^{n}[k(k-1)\times 6]$$

当 $n=4$ 时，代入可得理论最大调用数 $=72\times 36\times 12=31{,}104$ 次。

3. 实际运算中，许多路径会因找到解而提前终止，且不同选择顺序可能抵达重复数字集合，实际执行次数远小于此上限。

4. 每次递归的核心操作为向量复制和遍历，开销为 $O(k)$。总时间复杂度可视为 $O(n!^2\cdot 6^n)$ 级别，对于固定 $n=4$ 完全可视为常数时间，运行瞬间完成。

（二）空间复杂度

1. 递归深度最大为 $n-1$（本题深度 3），每层需保存当前数字向量 nums 的副本，空间为 $O(n^2)$（由于每层复制一次大小递减的向量，累积约 $O(n^2)$ 辅助空间）。

2. 对于 $n=4$，辅助空间极小，可视为 $O(1)$。

3. 无其他额外大型数据结构，整体空间复杂度为 $O(n^2)$，常数级存储。

3 复原 IP 地址

3.1 问题分析

1. 问题描述：给定一个仅由数字组成的字符串 s，需要在不改变字符顺序的前提下，插入恰好三个点 "."，将字符串分割成四个整数部分，使得每一部分满足：**（1）整数大小在 0 到 255 之间；（2）不能含有前导零（除非该部分就是单个数字 "0"）。**要求返回所有可能的合法 IP 地址。

2. 输入格式：一个字符串 s，如 "25525511135"。

3. 输出要求：一个字符串向量，包含所有可能的有效 IP 地址。

4. 示例：输入 "25525511135" → 输出 ["255.255.11.135","255.255.111.35"]。

5. 问题关键点

（1）分割数量固定为 4 段。

（2）每段长度受数值范围（≤255）和无前导零的限制，因此每段长度为 1 到 3。

（3）原字符串长度必须在 4 到 12 之间，否则无解。

3.2 算法思想

核心思路：回溯逐段分割

1. 状态定义：递归函数 backtrack(s, count, start, path, ans) 中：

count：当前已分割出的合法段数（0~4）

start：当前待分割子串的起始索引

path：存储当前已分割的各段字符串

ans：收集最终合法 IP 地址的结果向量

2. 终止条件：当 count == 4 时：

若 start == s.size()，说明刚好用完所有字符，将 path 中的四段用 "." 连接，加入 ans。否则直接返回（已分割完 4 段但字符有剩余，无效）。

3. 搜索与剪枝：从 start 开始尝试截取长度 1 到 3 的子串（且 i < s.size()）：

（1）调用 isvalid(s, start, i) 检查子串 s[start...i] 是否合法。

（2）合法条件（isvalid）：

① 如果子串长度大于 1，但首字符为 '0'，则包含前导零，非法；

② 如果子串对应的整数 > 255，非法。

（3）若合法，则将该子串加入 path，递归处理下一段：backtrack(s, count+1, i+1, path, ans)。

（4）递归返回后撤销选择（path.pop_back()），尝试下一种分割长度。

4. 回溯特性：通过“选择—递归—撤销”的经典回溯模式，能系统地遍历所有可能的分割方案，并在违反约束时及时剪枝。由于最多只有 4 段、每段最多 3 个字符，搜索空间非常小。

3.3 算法伪代码

```
函数 isvalid(s, start, end):
    如果 s[start] == '0' 且 end > start:
        返回 false          // 前导零
    如果 stoi(s.substr(start, end-start+1)) > 255:
        返回 false          // 数值超出范围
    返回 true
```

```
函数 backtrack(s, count, start, path, ans):
    如果 count == 4:
        如果 start == s.size():
            ip = path[0] + "." + path[1] + "." + path[2] + "." + path[3]
            ans.push_back(ip)
        返回

    对于 i 从 start 到 min(s.size()-1, start+2):
        如果 isvalid(s, start, i):
            path.push_back(s.substr(start, i-start+1))
            backtrack(s, count+1, i+1, path, ans)
            path.pop_back()

函数 restoreIPaddresses(s):
    path = 空向量
    ans = 空向量
    backtrack(s, 0, 0, path, ans)
    返回 ans

主程序:
    s = "25525511135"
    res = restoreIPaddresses(s)
    输出 res 中的每一个 IP
```

3.4 复杂度分析

设字符串长度为 n。由于有效 IP 地址每段最少 1 位、最多 3 位，总共 4 段，因此 n 的取值范围为 $[4, 12]$，可视为常数。

（一）时间复杂度

1. 递归深度固定为 4。

2. 每个阶段最多尝试 3 种分割长度（1、2、3），因此递归树的分支数 ≤ 3。

3. 最坏情况下，枚举所有可能的 $3^4 = 81$ 种分割方式。

4. 每次递归中，isvalid 检查子串并可能将其转换为整数（stoi），子串长度 ≤ 3，操作为 $O(1)$。

5. 构造最终 IP 字符串并进行比较的开销也是 $O(1)$。

6. 因此总时间复杂度为 $O(3^4) = O(81)$，即常数级。对于更长输入（若扩展到更一般情况），复杂度为 $O(3^4)$，实际运行极快。

（二）空间复杂度

1. 递归调用栈深度为 4，占用 $O(1)$ 栈空间。

2. path 存储最多 4 段字符串，每段长度 ≤ 3，空间 $O(1)$。

3. ans 存储所有合法结果，最坏情况下结果数量为 81 个（实际远少，约十几到几十），每个 IP 长度 ≤ 15，占用空间 $O(1)$（相对于 n 为常数）。

4. 因此整体空间复杂度为 $O(1)$。

4 分割回文串

4.1 问题分析

1. 问题描述：给定一个仅由小写英文字母组成的字符串 s，要求将 s 分割成若干个子串，使得每个子串都是回文串。返回所有可能的分割方案。每个方案是一个字符串列表，顺序拼接后等于原字符串。

2. 输入格式：一个字符串 s，长度范围 $1 \leq n \leq 16$，例如 "aab"。

3. 输出要求：所有满足条件的分割方案，以二维列表形式输出。方案内部子串顺序与在原串中出现的顺序一致，不同方案之间的顺序任意。

4. 问题关键点

（1）分割数量不固定，每个位置可以选择切分或不切分，因此构成一个决策树。

（2）必须保证每一步切出的子串是回文串，否则立即剪枝。

（3）字符串长度最大为 16，直接穷举所有分割方案并检验回文性质是可行的。

（4）可通过预处理提前计算任意子串是否为回文，避免重复判断，但原题规模较小，也可在回溯中实时判断。

4.2 算法思想

核心思路：回溯法 + 回文检验

1. 问题建模：将字符串 s 看作字符序列，从位置 0 开始，每次选择一个结束位置 end（从 start 到末尾），使得子串 s[start...end] 是回文串。将该子串加入当前解路径，然后对剩余字符串 s[end+1...] 进行递归处理。

2. 终止条件：当 start == s.size() 时，说明已经遍历完整个字符串，当前路径中的子串序列构成一个合法的分割方案，将其加入结果集。

3. 选择与剪枝：对当前起始位置 start，枚举所有可能的结束位置 end（start 到 n-1）：

若 s[start...end] 是回文串，则添加该子串到路径，并递归调用 backtrack(s, end+1, path, result)；若子串不是回文，直接跳过，实现剪枝。

4. 回文判断：用双指针法在 $O(k)$ 时间内判断一个长度为 k 的子串是否为回文。由于 n 很小，频繁判断的开销可以接受。

5. 回溯特性：每次递归深入前做出选择，返回后撤销选择（弹出子串），继续尝试下个结束位置。这样能系统地枚举所有符合回文约束的分割组合，不会遗漏也不会重复。

4.3 算法伪代码

```
函数 isPalindrome(s, left, right):
    当 left < right:
        如果 s[left] != s[right]: 返回 false
        left++, right--
    返回 true

函数 backtrack(s, start, path, result):
    如果 start == s.size():
        result.push_back(path)
        返回

    对于 end 从 start 到 s.size()-1:
        如果 isPalindrome(s, start, end):
```

```
        path.push_back(s.substr(start, end-start+1))
        backtrack(s, end+1, path, result)
        path.pop_back()

函数 partition(s):
    result = 空列表
    path = 空列表
    backtrack(s, 0, path, result)
    返回 result
```

4.4 复杂度分析

设字符串长度为 n，题目中 $n \leq 16$。

（一）时间复杂度

1. 回溯枚举部分：字符串的每个间隔位置都可以选择切分或不切分，因此理论上存在 2^{n-1} 种不同的分割方式（相当于在 n-1 个空隙中放置隔板）。最坏情况下，需要检查所有分割方案。

2. 回文判断部分：平均子串长度约 $O(n)$，每次判断为 $O(n)$，总复杂度约为 $O(n \cdot 2^n)$。

（二）空间复杂度

1. 递归调用栈深度最大为 n（每个字符单独成段），所需栈空间为 $O(n)$。

2. 存储路径 path 和结果 result：其中 path 最多容纳 n 个子串，空间 $O(n)$。result 存储所有合法分割方案，方案数量在最坏情况下（如全相同字符 "aaaa..."）可能达到 2^{n-1} 个，每个方案长度为 $O(n)$，因此空间复杂度为 $O(n \cdot 2^n)$。

5 删除无效的括号

5.1 问题分析

1. 问题描述：给定一个由小写字母和括号 '('、')' 组成的字符串 s，要求删除**最少数量**的无效括号，使得最终的字符串是一个有效的括号字符串（即左右括号匹配且顺序正确）。返回所有可能的结果，顺序任意。

2. 输入格式：一个字符串 s，长度范围 $1 \leq s.length \leq 25$。

3. 输出要求：一个字符串向量，包含所有删除最少括号后得到的有效字符串，不能有重复。

4. 问题关键点

（1）首先需要确定最少删除数量：必须删除的不匹配左括号和右括号。

（2）在回溯中，对每个括号有两种选择：保留或删除（但受剩余可删除数量的限制）。

（3）保留右括号时，必须满足当前路径中左括号数大于右括号数，否则会产生无效前缀。

（4）去重：可能存在不同删除位置导致相同的结果字符串（如连续相同的括号），使用 unordered_set 收集结果以去重。

（5）只需删除最少数量的括号，因此先计算最小删除数 lr、rr，再回溯。

5.2 算法思想

核心思路：两阶段策略——先计算最小删除数，再回溯构造所有有效串

阶段一：计算最少待删除括号数 lr 和 rr

遍历原字符串：

1. 遇到 '('：lr++（假设它是多余的左括号）

2. 遇到 ')'：

① 若 lr > 0，说明可以跟一个左括号匹配，执行 lr--

② 否则，该右括号多余，执行 rr++

3. 最终 lr 表示至少需要删除的左括号数量，rr 表示至少需要删除的右括号数量。

阶段二：带回溯的构造

定义递归函数 backtrack(s, i, lc, rc, lr, rr, path, ans)：

i：当前处理字符的位置

lc, rc：当前已构造路径中左括号和右括号的计数

lr, rr：剩余尚可删除的左括号和右括号数量

path：当前构造的字符串

ans：存储最终去重结果的 unordered_set

回溯规则：

1. 终止条件：i == s.size()

若 lr == 0 且 rr == 0 且 lc == rc（删除了恰好最少数量的括号，且括号完全匹配），则将 path 插入 ans。

2. 当前字符为字母（非括号）：直接保留，加入 path，递归到 i+1，然后撤销加入。

3. 当前字符为 '('：

若 lr > 0：可选择删除该左括号（不加入 path，lr 减 1，递归 i+1）。

选择保留该左括号：加入 path，lc 加 1，递归 i+1，然后撤销。

4. 当前字符为 ')'：

若 rr > 0：可选择删除该右括号（不加入 path，rr 减 1，递归 i+1）。

若 lc > rc：当前左括号多于右括号，可保留该右括号：加入 path，rc 加 1，递归 i+1，然后撤销。（lc > rc 条件保证了不会出现右括号提前多余左括号的情况）

去重：由于可能在不同位置上删除相同的括号而得到相同的结果串，使用 unordered_set<string> 自动去重，最后转为 vector 返回。

5.3 算法伪代码

```
函数 removeInvalidParentheses(s):
    lr = 0, rr = 0
    遍历 s 中的每个字符 c:
        如果 c == '(':
            lr = lr + 1
        否则如果 c == ')':
            如果 lr > 0:
                lr = lr - 1
            否则:
                rr = rr + 1

    path = 空字符串
    ans = 空无序集合
    backtrack(s, 0, 0, 0, lr, rr, path, ans)
```

```
    返回 vector<string>(ans.begin(), ans.end())

函数 backtrack(s, i, lc, rc, lr, rr, path, ans):
    如果 i == s.size():
        如果 lr == 0 且 rr == 0 且 lc == rc:
            ans.insert(path)
        返回

    c = s[i]
    如果 c 不等于 '(' 且 c 不等于 ')':
        path.push_back(c)
        backtrack(s, i+1, lc, rc, lr, rr, path, ans)
        path.pop_back()
    否则如果 c == '(':
        如果 lr > 0:
            backtrack(s, i+1, lc, rc, lr-1, rr, path, ans)     // 删除 '('
        path.push_back('(')
        backtrack(s, i+1, lc+1, rc, lr, rr, path, ans)         // 保留 '('
        path.pop_back()
    否则 (c == ')'):
        如果 rr > 0:
            backtrack(s, i+1, lc, rc, lr, rr-1, path, ans)     // 删除 ')'
        如果 lc > rc:
            path.push_back(')')
            backtrack(s, i+1, lc, rc+1, lr, rr, path, ans)    // 保留 ')'
            path.pop_back()
```

5.4 复杂度分析

设字符串长度为 n，题目中 $n \leq 16$。

（一）时间复杂度

1. 预处理计算 lr、rr：一次线性扫描 $\boldsymbol{O(n)}$。

2. 回溯搜索：每个括号字符面临保留/删除两种选择，理论搜索空间为 $\boldsymbol{O(2^n)}$。但由于以下剪枝策略，实际探索的节点远少于最坏情况：

① 受 lr 和 rr 上限约束，不可能删除超过最小必需数量的括号；

② 右括号只有在 lc > rc 时才能保留，提前剪掉了大量无效前缀。

在最坏情况下（例如全为括号且需要大量删除），时间复杂度仍为指数级 $\boldsymbol{O(2^n)}$，

3. 每个叶子节点还需要将字符串插入 unordered_set，平均 $\boldsymbol{O(1)}$ 到 $\boldsymbol{O(n)}$，整体影响不大。

（二）空间复杂度

1. 递归调用栈深度最大为 n，因此栈空间 $O(n)$。

2. 构造路径 path 长度最多为 n，空间 $O(n)$。

3. ans 存储所有去重后的结果，结果数量在最坏情况下可能达到 $O(2^n)$ 级别，但每个字符串长度为 $O(n)$，因此存储结果的空间为 $O(n \cdot 2^n)$。